```
end
```

The *nesting* at any given place is the collection of enclosing nested class and module objects outwards. The nesting at any given place can be inspected with `Module.nesting`. For example, in the previous example, the nesting at (1) is

```
[XML::SAXParser, XML]
```

It is important to understand that the nesting is composed of class and module *objects*, it has nothing to do with the constants used to access them, and is also unrelated to their names.
For instance, while this definition is similar to the previous one:

```
class XML::SAXParser
  # (2)
end
```

the nesting in (2) is different:

```
[XML::SAXParser]
```

`XML` does not belong to it.
We can see in this example that the name of a class or module that belongs to a certain nesting does not necessarily correlate with the namespaces at the spot.

Even more, they are totally independent, take for instance

```
module X
  module Y
  end
end

module A
  module B
  end
end

module X::Y
  module A::B
    # (3)
  end
end
```

The nesting in (3) consists of two module objects:

```
[A::B, X::Y]
```

So, it not only doesn't end in `A`, which does not even belong to the nesting, but it also contains `X::Y`, which is independent from `A::B`.
The nesting is an internal stack maintained by the interpreter, and it gets modified according to these rules:

- The class object following a `class` keyword gets pushed when its body is executed, and popped after it.
- The module object following a `module` keyword gets pushed when its body is executed, and popped after it.
- A singleton class opened with `class << object` gets pushed, and popped later.
- When `instance_eval` is called using a string argument, the singleton class of the receiver is pushed to the nesting of the eval'ed code. When `class_eval` or `module_eval` is called using a string argument, the receiver is pushed to the nesting of the eval'ed code.